xv6 VFS 里的对象：谁指向谁

实线箭头 = 真指针；虚线 = 按编号查表（dev 是 supers[] 下标，inum 是 inode 号）；点线 = 复制一份函数表指针

struct proc（每个进程一份）

ofile[NOFILE]	打开的文件
cwd	当前目录
cwdpath	当前目录的绝对路径

fs_root（全局，命名空间的 "/"）

struct inode *fs_root	chroot 后指向新根

ftable.file[NFILE]：struct file

打开文件描述（open 一次一个，dup/fork 共享）

type	FD_INODE / FD_DEVICE / FD_PIPE …
ref	fd、dup、fork 的引用数
ip	对应的 inode
off	共享的读写偏移
major	FD_DEVICE：主设备号
f_op	这次打开用的操作表
private_data	驱动 open 时放的状态

struct file_operations（只读表）

管“打开以后的读写”，参数是 struct file *

xv6_file_fops / xv6_dir_fops	xv6fs
ext2_file_fops / ext2_dir_fops	ext2
pathfs_file_fops / pathfs_dir_fops	procfs fat32 netfs
chrdevs[major].fops	字符设备驱动

字段：fread fwrite read_at write_at fsync poll ioctl mmap open release …

FD_DEVICE 的 f_op 由 chrdev_open() 按 major 取

open：f->f_op = ip->i_fop（file_inode_setup）

ip

file_systems 链表：struct fs_type

一种文件系统类型一项，启动时 register_filesystem()

name	"xv6fs" "ext2" "procfs" …
get_sb	source → dev 号
fill_super	读该 dev 的超级块，登记 supers[dev]
next	下一种类型

fill_super 填 supers[dev]

get_sb 只在 getsuper(dev)==0 时调它

itable.inode[NINODE=50]：struct inode

inode 缓存；身份是 (dev, inum)，ref>0 才算占用

dev	属于哪个文件系统实例
inum	该文件系统里的 inode 号
ref	引用数，0 = 空槽可复用
iop	这个 inode 用的 inode_ops
i_fop	open 时给 file 的默认 f_op
valid	已从盘上读入？
type major minor nlink size	盘上 inode 的副本
addrs[NDIRECT+1]	xv6fs：数据块号
path[MAXPATH]	按路径的文件系统：路径
i_rwsem	读写信号量

struct inode_ops

管“名字和元数据”，参数是 struct inode *

xv6_iops	引导：read_inode evict
xv6_dir_iops	+ lookup create unlink link rename
xv6_file_iops	+ truncate
xv6_device_iops	read_inode evict
ext2_iops	ext2 的同一组
pathfs_iops	procfs fat32 netfs

字段：read_inode evict lookup create unlink link rename truncate readlink

getsuper(ip->dev)

supers[NSUPER=24]：struct super_block

一个已挂载的文件系统实例一项，下标 = dev

dev / type	4 "xv6fs"、5 "ext2"、7、8、20 …
iop	引导用的 inode_ops
rootino	根目录的 inode 号（1 或 2）
readonly logged pathbased	挂载属性
priv	文件系统私有（ext2 实例）
put_super	umount 时收尾

mp

addrs[] → 块

iget：ip->iop = sb->iop（引导表）；read_inode

读出 type 后换成 dir/file/device 的 iops 和 i_fop

follow_mount：getsuper(m->dev)

→ iget(dev, rootino) 取新根

iget(dev, inum)

mtab.m[NFSMOUNT=8]：struct fsmount

挂载表：只记“哪个目录被哪个 dev 的根盖住”

mp	被盖住的目录（持一个引用）
dev	盖上去的文件系统
fstype source	"ext2" "/dev/root"
path	挂载点路径，只供显示和 umount
unmounting	卸载中，挡住新的查找

目录的数据块（xv6 没有 dentry）

struct dirent { ushort inum; char name[30]; }，32 字节一项

inum 1 "."
inum 1 ".."
inum 2 "bin"
inum 54 "dev" 被 devtmpfs 盖住
…

lookup 每次都线性扫描目录块，得到 inum 后 iget(dev, inum)；名字不缓存

和 Linux 的对应

file_system_type	fs_type 链表
super_block（链表）	supers[dev] 数组
struct mount + hash	mtab.m[] 线性扫描
inode cache	itable.inode[]
dentry / dcache	无：目录块 + dirlookup
inode->i_op	ip->iop
inode->i_fop / file->f_op	ip->i_fop / f->f_op